02157 Functional Programming
Week 1:
Introduction and Getting Started

Sebastian Mödersheim
Based on slides by Michael Reichhardt Hansen

September 4, 2026

WELCOME to
02157 Functional Programming

Teacher: Sebastian Mödersheim, DTU Compute samo@dtu.dk

Teaching assistants:

Andrei Chiosa
Elias Haynie-Gay
Emil Breinholt
Jacob Eskild Jensen
Jóhannes Sigvardsen
Teaching Assistant Plus
on our DTU Learn page/classlist

Homepage: www.compute.dtu.dk/courses/02157

These Slides

I have taken over this course last year.

These slides are based on slides by Michael R. Hansen
who taught the course in the previous years.

By default you should assume that

- everything great on these slides is due to Michael and
- everything that sucks is probably my fault. :-)

Please let me know if you find errors or if anything is unclear.

Functional Programming (FP) is Awesome

- Mathematical way of programming
  - ★ abstract, succinct, precise, no mess
  - ★ easier to write, to read, and to extend
  - ★ speed of development, less debugging time
  - ★ quality of code: fewer bugs and security problems

Functional Programming (FP) is Awesome

- Mathematical way of programming
  - ★ abstract, succinct, precise, no mess
  - ★ easier to write, to read, and to extend
  - ★ speed of development, less debugging time
  - ★ quality of code: fewer bugs and security problems
- Programming Language Concepts: FP is extremely helpful for:
  - ★ compiler design and theoretical computer science (02141 Computer Science Modeling, 02247 Compiler Construction)
  - ★ formal verification (02242 Program Analysis, 02244 Logic for Security)
  - ★ theorem proving (02256 Automated Reasoning)

Functional Programming (FP) is Awesome

- Mathematical way of programming
  - ★ abstract, succinct, precise, no mess
  - ★ easier to write, to read, and to extend
  - ★ speed of development, less debugging time
  - ★ quality of code: fewer bugs and security problems
- Programming Language Concepts: FP is extremely helpful for:
  - ★ compiler design and theoretical computer science (02141 Computer Science Modeling, 02247 Compiler Construction)
  - ★ formal verification (02242 Program Analysis, 02244 Logic for Security)
  - ★ theorem proving (02256 Automated Reasoning)
- Transfer: An FP background helps also when using a conventional language
  - ★ You can use FP concepts to systematically write better imperative code
  - ★ Conventional programming languages incrementally adopt features from FP
    - ▶ e.g. higher-order functions, algebraic data types, automatic type inference, ...

Functional Programming (FP) is Awesome

- Mathematical way of programming
  - ★ abstract, succinct, precise, no mess
  - ★ easier to write, to read, and to extend
  - ★ speed of development, less debugging time
  - ★ quality of code: fewer bugs and security problems
- Programming Language Concepts: FP is extremely helpful for:
  - ★ compiler design and theoretical computer science (02141 Computer Science Modeling, 02247 Compiler Construction)
  - ★ formal verification (02242 Program Analysis, 02244 Logic for Security)
  - ★ theorem proving (02256 Automated Reasoning)
- Transfer: An FP background helps also when using a conventional language
  - ★ You can use FP concepts to systematically write better imperative code
  - ★ Conventional programming languages incrementally adopt features from FP
    - ▶ e.g. higher-order functions, algebraic data types, automatic type inference, ...
- Senior Programmers
  - ★ AI is getting more and more powerful in programming
  - ★ We need human supervisors of AI who can
    - ▶ Understand programming concepts, planning, structuring
    - ▶ Understand the produced code and its quality
    - ▶ Verify/prove that solutions are correct and secure

Functional Programming (FP) is Awesome

- Mathematical way of programming
  - ★ abstract, succinct, precise, no mess
  - ★ easier to write, to read, and to extend
  - ★ speed of development, less debugging time
  - ★ quality of code: fewer bugs and security problems
- Programming Language Concepts: FP is extremely helpful for:
  - ★ compiler design and theoretical computer science (02141 Computer Science Modeling, 02247 Compiler Construction)
  - ★ formal verification (02242 Program Analysis, 02244 Logic for Security)
  - ★ theorem proving (02256 Automated Reasoning)
- Transfer: An FP background helps also when using a conventional language
  - ★ You can use FP concepts to systematically write better imperative code
  - ★ Conventional programming languages incrementally adopt features from FP
    - ▶ e.g. higher-order functions, algebraic data types, automatic type inference, ...
- Senior Programmers
  - ★ AI is getting more and more powerful in programming
  - ★ We need human supervisors of AI who can
    - ▶ Understand programming concepts, planning, structuring
    - ▶ Understand the produced code and its quality
    - ▶ Verify/prove that solutions are correct and secure
- Invest the Time!
  - ★ It requires a lot of time to become good at FP
  - ★ You need to do exercises - a lot!
  - ★ It will be an excellent investment: you will get a pro programmer!
  - ★ It will be a lot of fun!

Plan for Today

1. The concepts of functional programming
2. Organizational matters
3. Introduction to F#
4. Lists

The Concepts of Functional Programming

Note: We will sometimes work on the blackboard. I will take pictures after the lecture and upload them to DTU Learn.

Example: contrasting 3 ways to write exponentiation:

Mathematical Definition	Functional Program	Imperative Program

Summary:

- Functional programming is close to mathematical descriptions.
  - ★ Recursion instead of loops
  - ★ Variables are like variables in math
- It is often much easier to write correct programs, prove their correctness, and to read and extend them.
- It is often easier to write asymptotically efficient programs.
- We will see how to transform recusion into loops.
- Functional programming knowledge can help you
  even when you are forced to use an imperative language.

There is no magic

It is possible to understand everything:

There is no magic

It is possible to understand everything:

- The syntax (notation) of the programming language

There is no magic

It is possible to understand everything:

- The syntax (notation) of the programming language
- The semantics (meaning) of programs

There is no magic

It is possible to understand everything:

- The syntax (notation) of the programming language
- The semantics (meaning) of programs
- The evaluation of programs

There is no magic

It is possible to understand everything:

- The syntax (notation) of the programming language
- The semantics (meaning) of programs
- The evaluation of programs
- The properties of programs

There is no magic

It is possible to understand everything:

- The syntax (notation) of the programming language
- The semantics (meaning) of programs
- The evaluation of programs
- The properties of programs

Functional programming is a simple setting supporting

- declaration of clear, concise programs at a high level of abstraction
- understanding and analysis of programs

due to the basis on mathematical functions (no side-effects)

Some functional programming background

- The λ-calculus was introduced around 1930 by Church and Kleene when investigating function definition, function application, recursion and computable functions. For example, $f(x) = x + 2$ is represented by $\lambda x.x + 2$.
  - ★ We will look in more detail at the λ-calculus in one lecture.

Some functional programming background

- The λ-calculus was introduced around 1930 by Church and Kleene when investigating function definition, function application, recursion and computable functions. For example, $f(x) = x + 2$ is represented by $\lambda x.x + 2$.
  - ★ We will look in more detail at the λ-calculus in one lecture.
- The untyped functional-like programming language LISP was developed by McCarthy in the late 1950s.

Some functional programming background

- The λ-calculus was introduced around 1930 by Church and Kleene when investigating function definition, function application, recursion and computable functions. For example, $f(x) = x + 2$ is represented by $\lambda x.x + 2$.
  - ★ We will look in more detail at the λ-calculus in one lecture.
- The untyped functional-like programming language LISP was developed by McCarthy in the late 1950s.
- Functional languages with a strong type system like ML (by Milner) were introduced in the 1970s.

Some functional programming background

- The λ-calculus was introduced around 1930 by Church and Kleene when investigating function definition, function application, recursion and computable functions. For example, $f(x) = x + 2$ is represented by $\lambda x.x + 2$.
  - ★ We will look in more detail at the λ-calculus in one lecture.
- The untyped functional-like programming language LISP was developed by McCarthy in the late 1950s.
- Functional languages with a strong type system like ML (by Milner) were introduced in the 1970s.
- Functional languages (SML, Haskell, OCAML, F#, ...) have now applications far away from their origin: Compilers, Artificial Intelligence, Web-applications, Financial sector, ...

Some functional programming background

- The λ-calculus was introduced around 1930 by Church and Kleene when investigating function definition, function application, recursion and computable functions. For example, $f(x) = x + 2$ is represented by $\lambda x.x + 2$.
  - ★ We will look in more detail at the λ-calculus in one lecture.
- The untyped functional-like programming language LISP was developed by McCarthy in the late 1950s.
- Functional languages with a strong type system like ML (by Milner) were introduced in the 1970s.
- Functional languages (SML, Haskell, OCAML, F#, ...) have now applications far away from their origin: Compilers, Artificial Intelligence, Web-applications, Financial sector, ...
- Declarative aspects are now sneaking into "main stream languages"

Some functional programming background

- The λ-calculus was introduced around 1930 by Church and Kleene when investigating function definition, function application, recursion and computable functions. For example, $f(x) = x + 2$ is represented by $\lambda x.x + 2$.
  - ★ We will look in more detail at the λ-calculus in one lecture.
- The untyped functional-like programming language LISP was developed by McCarthy in the late 1950s.
- Functional languages with a strong type system like ML (by Milner) were introduced in the 1970s.
- Functional languages (SML, Haskell, OCAML, F#, ...) have now applications far away from their origin: Compilers, Artificial Intelligence, Web-applications, Financial sector, ...
- Declarative aspects are now sneaking into "main stream languages"
- Functional programming should be a mandatory element of every BSc. education in Computer Science according to ACM's and IEEE's curricula recommendations, 2013.

Have a look at

- http://homepages.inf.ed.ac.uk/wadler/realworld/
- https://fsharp.org/testimonials/

concerning use of functional programming in the "real world".

Part 2 – Practical Matters

- General information: http://courses.compute.dtu.dk/02157

Part 2 – Practical Matters

- General information: http://courses.compute.dtu.dk/02157
- Practical Information:
  http://courses.compute.dtu.dk/02157/PracticalInfo.html

Part 2 – Practical Matters

- General information: http://courses.compute.dtu.dk/02157
- Practical Information:
  http://courses.compute.dtu.dk/02157/PracticalInfo.html
- Course plan:
  http://courses.compute.dtu.dk/02157/plan.html

Part 2 – Practical Matters

- General information: http://courses.compute.dtu.dk/02157

- Practical Information:
  http://courses.compute.dtu.dk/02157/PracticalInfo.html

- Course plan:
  http://courses.compute.dtu.dk/02157/plan.html

- Exam form: Written exam - 4 hours
  Aid: all aid - without internet access
  meaning that in addition to written aids, it is also permitted to bring a laptop, tablet, and similar devices. The use of **generative AI is not allowed**.
  Exam date: 16. December 2026 (E4B)

Course Structure

- Weekly lectures Fridays 8:00-10:00
  in 308 auditorium 012
- Weekly exercises with fantastic TAs Fridays 10:00-12:00
  in 308 group rooms 001,009,017,127

Course design is based on an evenly distributed workload and "steady progress" throughout the semester

Course Structure

- Weekly lectures Fridays 8:00-10:00
  in 308 auditorium 012
- Weekly exercises with fantastic TAs Fridays 10:00-12:00
  in 308 group rooms 001,009,017,127

Course design is based on an evenly distributed workload and "steady progress" throughout the semester

This course had about 30% failure rate in the exam for many years.

Course Structure

- Weekly lectures Fridays 8:00-10:00
  in 308 auditorium 012
- Weekly exercises with fantastic TAs Fridays 10:00-12:00
  in 308 group rooms 001,009,017,127

Course design is based on an evenly distributed workload and "steady progress" throughout the semester

This course had about 30% failure rate in the exam for many years.
I would like to change that.

- FP is not easy, but everybody can do it!
- It requires that you invest the time
  - ★ Attend the lectures; if you cannot attend, see that you review the material (slides/book)
  - ★ Attend the exercises; you cannot learn it without doing it
  - ★ Ask the TAs and me questions
    - ▶ You can also hand in solutions to exercises to the TAs if you want feedback
  - ★ We often review in the lecture the exercises from the last week. Ask questions!
- Programming on paper:
  - ★ Instead of trying around in the interpreter, it is often helpful to develop it "dry" on paper first.

Course Structure

- Weekly lectures Fridays 8:00-10:00
  in 308 auditorium 012
- Weekly exercises with fantastic TAs Fridays 10:00-12:00
  in 308 group rooms 001,009,017,127

Course design is based on an evenly distributed workload and "steady progress" throughout the semester

This course had about 30% failure rate in the exam for many years.
I would like to change that.

- FP is not easy, but everybody can do it!
- It requires that you invest the time
  - ★ Attend the lectures; if you cannot attend, see that you review the material (slides/book)
  - ★ Attend the exercises; you cannot learn it without doing it
  - ★ Ask the TAs and me questions
    - ▶ You can also hand in solutions to exercises to the TAs if you want feedback
  - ★ We often review in the lecture the exercises from the last week. Ask questions!
- Programming on paper:
  - ★ Instead of trying around in the interpreter, it is often helpful to develop it "dry" on paper first.

You are always welcome to visit my office: Building 322, Room 212

Part 3 – Introduction to F#

- The interactive environment
- Values, expressions, types, patterns
- Declarations of values and recursive functions
- Binding, environment and evaluation
- Type inference

Main ingredients of F#

The Interactive Environment

```
2*3 + 4;;
val it : int = 10
```

The Interactive Environment

```
2*3 + 4;;
val it : int = 10
```

⇐ Input to the F# system

⇐ Answer from the F# system

The Interactive Environment

```
2*3 + 4;;
val it : int = 10
```

⇐ Input to the F# system

⇐ Answer from the F# system

- The *keyword* `val` indicates a value is computed
- The *integer* 10 is the computed value
- `int` is the *type* of the computed value
- The *identifier* `it` names the (last) computed value

The Interactive Environment

```
2*3 + 4;;
val it : int = 10
```

⇐ Input to the F# system

⇐ Answer from the F# system

- The *keyword* `val` indicates a value is computed
- The *integer* 10 is the computed value
- `int` is the *type* of the computed value
- The *identifier* `it` names the (last) computed value

Notion of a *Binding*: binding an identifier to a value.

$\texttt{it} \mapsto 10$ reads: "`it` is bound to 10"

Value Declarations

A value declaration has the form: `let` *identifier* = *expression*

```
let price = 25 * 5;;
```
⇐ A declaration as input

```
val price : int = 125
```
⇐ Answer from the F# system

The effect of a declaration is a binding: `price` $\mapsto$ 125

Value Declarations

A value declaration has the form: `let` *identifier* = *expression*

```
let price = 25 * 5;;
```
⇐ A declaration as input

```
val price : int = 125
```
⇐ Answer from the F# system

The effect of a declaration is a binding: `price` $\mapsto$ 125

Bound identifiers can be used in expressions and declarations, e.g.

```
let newPrice = 2*price;;
val newPrice : int = 250

newPrice > 500;;
val it : bool = false
```

Value Declarations

A value declaration has the form: `let` *identifier* = *expression*

```
let price = 25 * 5;;
```
⇐ A declaration as input

```
val price : int = 125
```
⇐ Answer from the F# system

The effect of a declaration is a binding: `price` $\mapsto$ 125

Bound identifiers can be used in expressions and declarations, e.g.

```
let newPrice = 2*price;;
val newPrice : int = 250

newPrice > 500;;
val it : bool = false
```

A collection of bindings

$$\begin{bmatrix} \texttt{price} & \mapsto & 125 \\ \texttt{newPrice} & \mapsto & 250 \\ \texttt{it} & \mapsto & \text{false} \end{bmatrix}$$

is called an environment

Function Declarations 1: `let f x = e`

- x is called the *formal parameter*
- the defining expression e is called the *body* of the declaration

Function Declarations 1: `let f x = e`

- x is called the *formal parameter*
- the defining expression e is called the *body* of the declaration

Declaration of the circle area function:

```
let circleArea r = System.Math.PI * r * r;;
```

- `System.Math` is a program library
- `PI` is an identifier (with type `float`) for π in `System.Math`

The type is automatically inferred in the answer:

```
val circleArea : float -> float
```

Note: functions are also considered as values (but cannot be printed)

Function Declarations 1: `let f x = e`

- x is called the *formal parameter*
- the defining expression e is called the *body* of the declaration

Declaration of the circle area function:

```
let circleArea r = System.Math.PI * r * r;;
```

- `System.Math` is a program library
- `PI` is an identifier (with type `float`) for π in `System.Math`

The type is automatically inferred in the answer:

```
val circleArea : float -> float
```

Note: functions are also considered as values (but cannot be printed)

Applications of the function (aka function call):

```
circleArea 1.0;; (* this is a comment *)
val it : float = 3.141592654

circleArea(3.2);; // A comment: optional brackets
val it : float = 32.16990877
```

`1.0` and `3.2` are also called *actual parameters*

Syntax, Type Checking, and Semantics

- The syntax of a programming language describes what sequences of characters consitute a correctly formed program,
  - ★ e.g. balancing of opening and closing parentheses.
- Type checking checks that all values (including functions) used in the program are indeed defined and used with the correct types,
  - ★ e.g. a function that expects an int cannot be called with a string.
- Semantics says what a syntactically correct, well-typed program means.
  - ★ e.g. `if x>0 then x else 2*x` with $[x \mapsto -3]$ evaluates to -6.
- In 02141 Computer Science Modelling you will learn how to formally describe syntax and semantics of programming languages.
- During the course, we will make such descriptions for reference, but not in all details formally. You will not be asked in the exam to describe syntax or semantics of F#.

The Syntax unlocked so far (1/2):

Constants

- `int` constants like $0, 1, -1, \ldots$
- `float` constants like 0.0, 3.14, ...
- Later: `string` constants like "*Hello*" and `bool` constants *true* and *false*

Identifiers

like `x` or `exp` are sequences of alphanumberic characters, i.e. letters and digits, where the first must be a letter.

Expressions

can be any of the following:

- A constant
- An identifier like x
- If e, e_1, $\ldots e_n$ are expressions and $\oplus$ is an infix operator (like $+$, $*$, $<$, $=$,...), then also the following are expressions:
  - ★ $e_1 \oplus e_2$
  - ★ `if` e `then` e_1 `else` e_2
  - ★ (e)
  - ★ $e_1\ e_2$
  - ★ $(e_1, \ldots, e_n)$

The Syntax unlocked so far (2/2):

Declarations

If e is an expression and f and $x_1, \ldots, x_n$ are identifiers, then the following are declarations:

- `let` $f = e$
- `let` $f\ (x_1, \ldots, x_n) = e$
  where the x_i may occur in e.

and the following are recursive declarations:

- `let rec` $f = e$
- `let rec` $f(x_1, \ldots, x_n) = e$

where also f may occur in e

Patterns

The power function defined earlier:

```
let rec power(x,n) =
  if (n=0) then 1.0
  else x * power(x,n-1)
```

Patterns

The power function defined earlier:

```
let rec power(x,n) =
  if (n=0) then 1.0
  else x * power(x,n-1)
```

An equivalent definition is using pattern matching:

```
let rec power(x,n) =
   match (x,n) with
   | (_,0)  -> 1.0                (* 1 *)
   | (x,n) -> x * power(x,n-1)    (* 2 *)
```

- The first pattern $(_, 0)$ is matches an arbitrary value for x but only 0 for n.
- The second pattern (x, n) matches any value (x, n).
- The second pattern is only tried if the first one does not match, thus it catches all other cases.

Patterns

Patterns

can be any of the following:

- A constant
- An identifier like x
- The wildcard _
- And if $p_1, \ldots p_n$ are patterns, then also $(p_1, \ldots, p_n)$

Expressions

An Expression can also be (in addition to what we allowed before):

$$\begin{array}{l}\texttt{match } e \texttt{ with} \\ \quad \mid p_1 \rightarrow e_1 \\ \quad \ldots \\ \quad \mid p_n \rightarrow e_n\end{array}$$

if $p_1, \ldots, p_n$ are patterns and $e, e_1, \ldots, e_n$ are expressions.

Examples:

Patterns

Patterns

can be any of the following:

- A constant
- An identifier like x
- The wildcard `_`
- And if $p_1, \ldots p_n$ are patterns, then also $(p_1, \ldots, p_n)$

Expressions

An Expression can also be (in addition to what we allowed before):

$$\begin{array}{l}\texttt{match } e \texttt{ with} \\ \quad \mid p_1 \rightarrow e_1 \\ \quad \ldots \\ \quad \mid p_n \rightarrow e_n\end{array}$$

if $p_1, \ldots, p_n$ are patterns and $e, e_1, \ldots, e_n$ are expressions.

Examples:

- Value 3.1 matches pattern `x` resulting in environment: $[x \mapsto 3.1]$

Patterns

Patterns

can be any of the following:

- A constant
- An identifier like x
- The wildcard `_`
- And if $p_1, \ldots p_n$ are patterns, then also $(p_1, \ldots, p_n)$

Expressions

An Expression can also be (in addition to what we allowed before):

$$\begin{array}{l}\texttt{match } e \texttt{ with} \\ \quad \mid p_1 \rightarrow e_1 \\ \quad \ldots \\ \quad \mid p_n \rightarrow e_n\end{array}$$

if $p_1, \ldots, p_n$ are patterns and $e, e_1, \ldots, e_n$ are expressions.

Examples:

- Value 3.1 matches pattern `x` resulting in environment: $[x \mapsto 3.1]$
- Value `true` matches pattern `true` resulting in environment $[\,]$

Patterns

Patterns

can be any of the following:

- A constant
- An identifier like x
- The wildcard _
- And if $p_1, \ldots p_n$ are patterns, then also $(p_1, \ldots, p_n)$

Expressions

An Expression can also be (in addition to what we allowed before):

$$\begin{array}{l}\texttt{match } e \texttt{ with} \\ \quad \mid p_1 \rightarrow e_1 \\ \quad \ldots \\ \quad \mid p_n \rightarrow e_n\end{array}$$

if $p_1, \ldots, p_n$ are patterns and $e, e_1, \ldots, e_n$ are expressions.

Examples:

- Value 3.1 matches pattern `x` resulting in environment: $[x \mapsto 3.1]$
- Value `true` matches pattern `true` resulting in environment $[\,]$
- The pair $(1, \texttt{true})$ matches pattern $(\texttt{x}, \texttt{y})$ resulting in environment $[x \mapsto 1, y \mapsto \mathit{true}]$

Match expressions

A match expression `match` e `with` is evaluated as follows:

$$
\begin{array}{l}
\texttt{match}\ e\ \texttt{with} \\
\quad \mid p_1 \rightarrow e_1 \\
\qquad\qquad \vdots \\
\quad \mid p_n \rightarrow e_n
\end{array}
$$

1. evaluate e to a value, say v
2. search for the first pattern p_i matching v
3. evaluate e_i in an environment enriched with the bindings from the pattern matching

Match expressions

A match expression `match` e `with` is evaluated as follows:

$$
\begin{array}{l}
\texttt{match } e \texttt{ with} \\
\quad \mid p_1 \rightarrow e_1 \\
\qquad \vdots \\
\quad \mid p_n \rightarrow e_n
\end{array}
$$

1. evaluate e to a value, say v
2. search for the first pattern p_i matching v
3. evaluate e_i in an environment enriched with the bindings from the pattern matching

If no pattern matches v, then the evaluation terminates with an error.

Example: Match on a pair

```
match (3+5, 3<5) with
| (0, _)     -> 0
| (n,false)  -> -n
| (n,_)      -> 2*n
```

Example: Match on a pair

```
match (3+5, 3<5) with
| (0, _)     -> 0
| (n,false)  -> -n
| (n,_)      -> 2*n
```

$\rightsquigarrow$

```
match (8, true) with
| (0, _)     -> 0
| (n,false)  -> -n
| (n,_)      -> 2*n
```

Example: Match on a pair

```
match (3+5, 3<5) with
| (0, _)     -> 0
| (n,false)  -> -n
| (n,_)      -> 2*n
```

$\rightsquigarrow$

```
match (8, true) with
| (0, _)     -> 0
| (n,false)  -> -n
| (n,_)      -> 2*n
```

$\rightsquigarrow$

```
match (8, true) with
| (n,false)  -> -n
| (n,_)      -> 2*n
```

Example: Match on a pair

```
match (3+5, 3<5) with
| (0, _)     -> 0
| (n,false)  -> -n
| (n,_)      -> 2*n
```

```
⇝  match (8, true) with
   | (0, _)     -> 0
   | (n,false)  -> -n
   | (n,_)      -> 2*n
```

```
⇝  match (8, true) with
   | (n,false)  -> -n
   | (n,_)      -> 2*n
```

```
⇝  match (8, true) with
   | (n,_)      -> 2*n
```

Example: Match on a pair

```
match (3+5, 3<5) with
| (0, _)     -> 0
| (n,false)  -> -n
| (n,_)      -> 2*n
```

$\rightsquigarrow$

```
match (8, true) with
| (0, _)     -> 0
| (n,false)  -> -n
| (n,_)      -> 2*n
```

$\rightsquigarrow$

```
match (8, true) with
| (n,false)  -> -n
| (n,_)      -> 2*n
```

$\rightsquigarrow$

```
match (8, true) with
| (n,_)      -> 2*n
```

$\rightsquigarrow$ (2*n,n $\mapsto$ 8) $\rightsquigarrow$ (16,n $\mapsto$ 8)

Evaluation. Example: `power(4.0, 2)`

```
let rec power(x,n) =
   match (x,n) with
   | (_,0) -> 1.0                    (* 1 *)
   | (x,n) -> x * power(x,n-1)       (* 2 *)
```

$$
\begin{array}{lll}
 & \texttt{power}(4.0, 2) & [] \\
\rightsquigarrow & \texttt{match}\ (\texttt{x}, \texttt{n})\ \texttt{with} & [\texttt{x} \mapsto 4.0, \texttt{n} \mapsto 2] \\
 & \mid (_, 0) \rightarrow 1.0 & \\
 & \mid (\texttt{x}, \texttt{n}) \rightarrow \texttt{x} * \texttt{power}(\texttt{x}, \texttt{n} - 1) & \\
\rightsquigarrow & \texttt{match}\ (4.0, 2)\ \texttt{with} & [\texttt{x} \mapsto 4.0, \texttt{n} \mapsto 2] \\
 & \mid (_, 0) \rightarrow 1.0 & \\
 & \mid (\texttt{x}, \texttt{n}) \rightarrow \texttt{x} * \texttt{power}(\texttt{x}, \texttt{n} - 1) & \\
\rightsquigarrow & \texttt{match}\ (4.0, 2)\ \texttt{with} & [\texttt{x} \mapsto 4.0, \texttt{n} \mapsto 2] \\
 & \mid (\texttt{x}, \texttt{n}) \rightarrow \texttt{x} * \texttt{power}(\texttt{x}, \texttt{n} - 1) & \\
\rightsquigarrow & \texttt{x} * \texttt{power}(\texttt{x}, \texttt{n} - 1) & [\texttt{x} \mapsto 4.0, \texttt{n} \mapsto 2] \\
\rightsquigarrow & 4.0 * \texttt{power}(4.0, 1) & [\texttt{x} \mapsto 4.0, \texttt{n} \mapsto 2]
\end{array}
$$

Similarly:
$\texttt{power}(4.0, 1) \rightsquigarrow 4.0 * \texttt{power}(4.0, 0)$
$\texttt{power}(4.0, 0) \rightsquigarrow 1.0.$

Types — every expression has a type $e : \tau$

Basic types:

	type name	example of values
Integers	`int`	27, 0, 15, 21000
Floats	`float`	27.3, 0.0, 48.21
Booleans	`bool`	true, false

Types — every expression has a type $e : \tau$

Basic types:

	type name	example of values
Integers	`int`	27, 0, 15, 21000
Floats	`float`	27.3, 0.0, 48.21
Booleans	`bool`	true, false

Pairs:

If $e_1 : \tau_1$ and $e_2 : \tau_2$

then $(e_1, e_2) : \tau_1 * \tau_2$ pair (tuple) type constructor

Types — every expression has a type $e : \tau$

Basic types:

	type name	example of values
Integers	`int`	27, 0, 15, 21000
Floats	`float`	27.3, 0.0, 48.21
Booleans	`bool`	true, false

Pairs:
If $e_1 : \tau_1$ and $e_2 : \tau_2$
then $(e_1, e_2) : \tau_1 * \tau_2$ pair (tuple) type constructor

Functions:
if $f : \tau_1$ `->` τ_2 and $a : \tau_1$ function type constructor
then $f(a) : \tau_2$

Types — every expression has a type $e : \tau$

Basic types:

	type name	example of values
Integers	`int`	27, 0, 15, 21000
Floats	`float`	27.3, 0.0, 48.21
Booleans	`bool`	true, false

Pairs:
If $e_1 : \tau_1$ and $e_2 : \tau_2$
then $(e_1, e_2) : \tau_1 * \tau_2$ pair (tuple) type constructor

Functions:
if $f : \tau_1$ `->` τ_2 and $a : \tau_1$ function type constructor
then $f(a) : \tau_2$

Examples:

```
(4.0, 2):  float*int
power:  float*int -> float
power(4.0, 2):  float
```

`*` has higher precedence than `->`

Type inference: `power`

```
let rec power (x,n) =
   match (x,n) with
   | (_,0) -> 1.0                    (* 1 *)
   | (x,n) -> x * power(x,n-1)       (* 2 *)
```

Type inference: `power`

```
let rec power (x,n) =
   match (x,n) with
   | (_,0) -> 1.0                    (* 1 *)
   | (x,n) -> x * power(x,n-1)       (* 2 *)
```

- The type of `power` must have the form: τ_1 `*` τ_2 `->` τ_3, because argument is a pair.

Type inference: `power`

```
let rec power (x,n) =
   match (x,n) with
   | (_,0) -> 1.0                    (* 1 *)
   | (x,n) -> x * power(x,n-1)       (* 2 *)
```

- The type of `power` must have the form: τ_1 `*` τ_2 `->` τ_3, because argument is a pair.
- τ_3 = `float` because `1.0:float` (Clause 1, function value.)

Type inference: `power`

```
let rec power (x,n) =
   match (x,n) with
   | (_,0) -> 1.0                    (* 1 *)
   | (x,n) -> x * power(x,n-1)       (* 2 *)
```

- The type of `power` must have the form: τ_1 `*` τ_2 `->` τ_3, because argument is a pair.
- τ_3 = `float` because `1.0:float` (Clause 1, function value.)
- τ_2 = `int` because `0:int` (Clause 1, pattern in position for `n`).

Type inference: `power`

```
let rec power (x,n) =
   match (x,n) with
   | (_,0) -> 1.0                    (* 1 *)
   | (x,n) -> x * power(x,n-1)       (* 2 *)
```

- The type of `power` must have the form: τ_1 `*` τ_2 `->` τ_3, because argument is a pair.
- τ_3 = `float` because `1.0:float` (Clause 1, function value.)
- τ_2 = `int` because `0:int` (Clause 1, pattern in position for `n`).
- `x*power(x,n-1):float`, because τ_3 = `float`.

Type inference: `power`

```
let rec power (x,n) =
   match (x,n) with
   | (_,0) -> 1.0                    (* 1 *)
   | (x,n) -> x * power(x,n-1)       (* 2 *)
```

- The type of `power` must have the form: τ_1 `*` τ_2 `->` τ_3, because argument is a pair.
- τ_3 = `float` because `1.0:float` (Clause 1, function value.)
- τ_2 = `int` because `0:int` (Clause 1, pattern in position for `n`).
- `x*power(x,n-1):float`, because τ_3 = `float`.
- multiplication can have

  `int*int -> int` or `float*float -> float`

  as types, but no "mixture" of `int` and `float`

Type inference: `power`

```
let rec power (x,n) =
   match (x,n) with
   | (_,0) -> 1.0                    (* 1 *)
   | (x,n) -> x * power(x,n-1)       (* 2 *)
```

- The type of `power` must have the form: τ_1 `*` τ_2 `->` τ_3, because argument is a pair.
- τ_3 = `float` because `1.0:float` (Clause 1, function value.)
- τ_2 = `int` because `0:int` (Clause 1, pattern in position for `n`).
- `x*power(x,n-1):float`, because τ_3 = `float`.
- multiplication can have

  `int*int -> int` or `float*float -> float`

  as types, but no "mixture" of `int` and `float`
- Therefore `x:float` and τ_1=`float`.

Type inference: `power`

```
let rec power (x,n) =
   match (x,n) with
   | (_,0) -> 1.0                    (* 1 *)
   | (x,n) -> x * power(x,n-1)       (* 2 *)
```

- The type of `power` must have the form: τ_1 `*` τ_2 `->` τ_3, because argument is a pair.
- τ_3 = `float` because `1.0:float` (Clause 1, function value.)
- τ_2 = `int` because `0:int` (Clause 1, pattern in position for `n`).
- `x*power(x,n-1):float`, because τ_3 = `float`.
- multiplication can have

  `int*int -> int` or `float*float -> float`

  as types, but no "mixture" of `int` and `float`
- Therefore `x:float` and τ_1=`float`.

The F# system determines the type `float*int -> float`

Type inference: `power`

```
let rec power (x,n) =
   match (x,n) with
   | (_,0) -> 1.0                    (* 1 *)
   | (x,n) -> x * power(x,n-1)       (* 2 *)
```

- The type of `power` must have the form: τ_1 `*` τ_2 `->` τ_3, because argument is a pair.
- τ_3 = `float` because `1.0:float` (Clause 1, function value.)
- τ_2 = `int` because `0:int` (Clause 1, pattern in position for `n`).
- `x*power(x,n-1):float`, because τ_3 = `float`.
- multiplication can have

  `int*int -> int` or `float*float -> float`

  as types, but no "mixture" of `int` and `float`
- Therefore `x:float` and τ_1=`float`.

The F# system determines the type `float*int -> float`

Note that this type works with the entire expression,
e.g. `n` $-$ 1 is of type `int`$= \tau_2$.

A higher-order version of the power function

We shall now look at a version of `power` $x\ n = x^n$ with the type

```
power: float -> (int -> float)
```

A higher-order version of the power function

We shall now look at a version of `power` x $n = x^n$ with the type

```
power: float -> (int -> float)
```

- the argument of power is the base x

A higher-order version of the power function

We shall now look at a version of `power` x $n = x^n$ with the type

```
power: float -> (int -> float)
```

- the argument of power is the base x
- and `power` x is the function that maps exponent n to x^n

A higher-order version of the power function

We shall now look at a version of `power` x $n = x^n$ with the type

```
power: float -> (int -> float)
```

- the argument of power is the base x
- and `power` x is the function that maps exponent n to x^n

The function may be evaluated in *stages*:

```
let pow2 = power 2.0;;

pow2 3;;
val it : float = 8.0
pow2 4;;
val it : float = 16.0
```

A higher-order version of the power function

We shall now look at a version of `power` x $n = x^n$ with the type

```
power: float -> (int -> float)
```

- the argument of power is the base x
- and `power` x is the function that maps exponent n to x^n

The function may be evaluated in *stages*:

```
let pow2 = power 2.0;;

pow2 3;;
val it : float = 8.0
pow2 4;;
val it : float = 16.0
```

This higher-order version of power is declared by

```
let rec power x n = match n with
    | 0 -> 1.0
    | _ -> x * power x (n-1);;
```

A higher-order version of the power function

We shall now look at a version of `power` x $n = x^n$ with the type

```
power: float -> (int -> float)
```

- the argument of power is the base x
- and `power` x is the function that maps exponent n to x^n

The function may be evaluated in *stages*:

```
let pow2 = power 2.0;;

pow2 3;;
val it : float = 8.0
pow2 4;;
val it : float = 16.0
```

This higher-order version of power is declared by

```
let rec power x n = match n with
    | 0 -> 1.0
    | _ -> x * power x (n-1);;
```

The value of the function is a function

An expression for anonymous functions

Expression

An expression can also be an anonymous function:

$$
\begin{array}{l}
\texttt{function} \\
\texttt{|}\ pat_1 \ \rightarrow\ e_1 \\
\quad \vdots \\
\texttt{|}\ pat_n \ \rightarrow\ e_n
\end{array}
$$

This allows you to "tabulate" argument-value pairs of a function, e.g.:

```
function
| 2  -> 28   // February
| 4  -> 30   // April
| 6  -> 30   // June
| 9  -> 30   // September
| 11 -> 30   // November
| _  -> 31;; // All other months
val it : int -> int = <fun:clo@17-2>
```

An expression for anonymous functions

Expression

An expression can also be an anonymous function:

```
function
| pat_1 -> e_1
  ⋮
| pat_n -> e_n
```

This allows you to "tabulate" argument-value pairs of a function, e.g.:

```
function
| 2  -> 28   // February
| 4  -> 30   // April
| 6  -> 30   // June
| 9  -> 30   // September
| 11 -> 30   // November
| _  -> 31;; // All other months
val it : int -> int = <fun:clo@17-2>

it 2;;
val it : int = 28
```

Another higher-order version of the power function

We now have another look at `power` x $n = x^n$ with the type

```
power: float -> (int -> float)
```

Another higher-order version of the power function

We now have another look at `power` x $n = x^n$ with the type

```
power: float -> (int -> float)
```

The following declaration explicitly reveals that `power` x is a function:

```
let rec power x =
   function
   | 0 -> 1.0
   | n -> x * power x (n-1);;
```

Booleans

Type name `bool`

Values `false`, `true`

Operator	Type	
`not`	`bool -> bool`	negation

```
not true = false
not false = true
```

Booleans

Type name `bool`

Values `false`, `true`

Operator	Type	
`not`	`bool -> bool`	negation

```
not true = false
not false = true
```

Expressions

e_1 && e_2	"conjunction $e_1 \wedge e_2$"
e_1 || e_2	"disjunction $e_1 \vee e_2$"

Booleans

Type name `bool`

Values `false`, `true`

Operator	Type	
`not`	`bool -> bool`	negation

```
not true = false
not false = true
```

Expressions

e_1 && e_2	"conjunction $e_1 \wedge e_2$"
e_1 || e_2	"disjunction $e_1 \vee e_2$"

— are lazily evaluated, e.g.

```
1<2 || 5/0 = 1
⇝ true
```

Precedence: && has higher than ||

Summary

- The interactive environment
- Values, expressions, types, patterns
- Declarations of values and recursive functions
- Binding, environment and evaluation
- Type inference
- higher-order functions

Part 4 - Introduction to Lists

- Lists: values and constructors
- Recursions following the structure of lists
- Polymorphism

- The list concept is a natural, built-in ingredient of functional languages

Lists

A list is a finite sequence of elements having the same type:

$[v_1; \ldots; v_n]$ ($[\,]$ is called the empty list)

Lists

A list is a finite sequence of elements having the same type:

$[v_1; \ldots; v_n]$ ($[]$ is called the empty list)

```
[2;3;6];;
val it : int list = [2; 3; 6]
```

Lists

A list is a finite sequence of elements having the same type:

$[v_1; \ldots; v_n]$ ($[]$ is called the empty list)

```
[2;3;6];;
val it : int list = [2; 3; 6]

["a"; "ab"; "abc"; ""];;
val it : string list = ["a"; "ab"; "abc"; ""]
```

Lists

A list is a finite sequence of elements having the same type:

$[v_1; \ldots; v_n]$ ($[]$ is called the empty list)

```
[2;3;6];;
val it : int list = [2; 3; 6]

["a"; "ab"; "abc"; ""];;
val it : string list = ["a"; "ab"; "abc"; ""]

[sin; cos];;
val it : (float->float) list = [<fun:...>; <fun:...>]
```

Lists

A list is a finite sequence of elements having the same type:

$[v_1; \ldots; v_n]$ ($[]$ is called the empty list)

```
[2;3;6];;
val it : int list = [2; 3; 6]

["a"; "ab"; "abc"; ""];;
val it : string list = ["a"; "ab"; "abc"; ""]

[sin; cos];;
val it : (float->float) list = [<fun:...>; <fun:...>]

[(1,true); (3,true)];;
val it : (int * bool) list = [(1, true); (3, true)]
```

Lists

A list is a finite sequence of elements having the same type:

$[v_1; \ldots; v_n]$ ($[]$ is called the empty list)

```
[2;3;6];;
val it : int list = [2; 3; 6]

["a"; "ab"; "abc"; ""];;
val it : string list = ["a"; "ab"; "abc"; ""]

[sin; cos];;
val it : (float->float) list = [<fun:...>; <fun:...>]

[(1,true); (3,true)];;
val it : (int * bool) list = [(1, true); (3, true)]

[[]; [1]; [1;2]];;
val it : int list list = [[]; [1]; [1; 2]]
```

List constructors

The formal definition of lists:

Definition

Lists can be **constructed** as follows:

- [] (the empty list)
- If x is a value and l is a list, then also

$$x :: l$$

is a list.

List constructors

The formal definition of lists:

Definition

Lists of type τ can be **constructed** as follows:

- [] (the empty list)
- If x is a value of type τ and l is a τ list, then also

$$x :: l$$

is a τ list.

List constructors

The formal definition of lists:

Definition

Lists of type τ can be **constructed** as follows:

- [] (the empty list)
- If x is a value of type τ and l is a τ list, then also

$$x :: l$$

is a τ list.

The notation $[x_1; x_2; \ldots; x_n]$ is just a shorthand for $x_1 :: x_2 :: \ldots :: x_n :: []$.

List constructors

The formal definition of lists:

Definition

Lists of type τ can be **constructed** as follows:

- [] (the empty list)
- If x is a value of type τ and l is a τ list, then also

$$x :: l$$

is a τ list.

The notation $[x_1; x_2; \ldots; x_n]$ is just a shorthand for $x_1 :: x_2 :: \ldots :: x_n :: []$.

The constructor :: is called *cons*

List constructors

The formal definition of lists:

Definition

Lists of type τ can be **constructed** as follows:

- [] (the empty list)
- If x is a value of type τ and l is a τ list, then also

$$x :: l$$

is a τ list.

The notation $[x_1; x_2; \ldots; x_n]$ is just a shorthand for $x_1 :: x_2 :: \ldots :: x_n :: []$.

The constructor :: is called *cons*

For a non-empty list $[x_1; x_2; \ldots; x_n]$, $n \geq 1$, we call

- x_1 the *head* and
- $[x_2; \ldots; x_n]$ the *tail*.

There are library functions `List.head` and `List.tail`.

Recursion on lists – a simple example

$$\texttt{suml}\ [x_1\,;x_2\,;\ldots;x_n] = \sum_{i=1}^{n} x_i = x_1 + x_2 + \cdots + x_n = x_1 + \sum_{i=2}^{n} x_i$$

Recursion on lists – a simple example

$$\texttt{suml}\,[x_1;x_2;\ldots;x_n] = \sum_{i=1}^{n} x_i = x_1 + x_2 + \cdots + x_n = x_1 + \sum_{i=2}^{n} x_i$$

Constructors are used in list patterns

```
let rec suml xs =
   match xs with
   | []       -> 0
   | x::tail -> x + suml tail;;
val suml : int list -> int
```

Recursion on lists – a simple example

$$\texttt{suml}\,[x_1;x_2;\ldots;x_n] = \sum_{i=1}^{n} x_i = x_1 + x_2 + \cdots + x_n = x_1 + \sum_{i=2}^{n} x_i$$

Constructors are used in list patterns

```
let rec suml xs =
    match xs with
    | []      -> 0
    | x::tail -> x + suml tail;;
val suml : int list -> int
```

```
    suml [1;2]
⇝   1 + suml [2]          (x ↦ 1 and tail ↦ [2])
⇝   1 + (2 + suml [])     (x ↦ 2 and tail ↦ [])
⇝   1 + (2 + 0)           (the pattern [] matches the value [])
⇝   1 + 2
⇝   3
```

Recursion follows the structure of lists

A polymorphic list function (I)

The function remove y xs gives the list obtained from xs by deleting every occurrence of y, e.g. `remove` $2\ [1;2;0;2;7] = [1;0;7]$.

Recursion is following the structure of the list:

```
let rec remove y xs =
   match xs with
   | []                -> []
   | x::tail when x=y  -> remove y tail
   | x::tail           -> x::remove y tail;;
```

A polymorphic list function (I)

The function remove y xs gives the list obtained from xs by deleting every occurrence of y, e.g. `remove` $2\ [1;2;0;2;7] = [1;0;7]$.

Recursion is following the structure of the list:

```
let rec remove y xs =
   match xs with
   | []                -> []
   | x::tail when x=y  -> remove y tail
   | x::tail           -> x::remove y tail;;
```

List elements can be of any type that supports equality

```
remove :  'a -> 'a list -> 'a list when'a : equality
```

A polymorphic list function (I)

The function remove y xs gives the list obtained from xs by deleting every occurrence of y, e.g. `remove` $2\ [1;2;0;2;7] = [1;0;7]$.

Recursion is following the structure of the list:

```
let rec remove y xs =
   match xs with
   | []                -> []
   | x::tail when x=y  -> remove y tail
   | x::tail           -> x::remove y tail;;
```

List elements can be of any type that supports equality

```
remove :  'a -> 'a list -> 'a list when'a : equality
```

- 'a is a *type variable*
- 'a : equality is a type constraint

A polymorphic list function (I)

The function remove y xs gives the list obtained from xs by deleting every occurrence of y, e.g. `remove` $2\ [1;2;0;2;7] = [1;0;7]$.

Recursion is following the structure of the list:

```
let rec remove y xs =
    match xs with
    | []                -> []
    | x::tail when x=y  -> remove y tail
    | x::tail           -> x::remove y tail;;
```

List elements can be of any type that supports equality

```
remove :  'a -> 'a list -> 'a list when'a : equality
```

- 'a is a *type variable*
- 'a : equality is a type constraint

The F# system infers the most general type for remove

A polymorphic list function (II)

- A type containing type variables is called a polymorphic type
- The remove function is called a polymorphic function.

```
remove : 'a -> 'a list -> 'a list when 'a : equality
```

The function has many forms, one for each instantiation of 'a:

A polymorphic list function (II)

- A type containing type variables is called a polymorphic type
- The remove function is called a polymorphic function.

```
remove : 'a -> 'a list -> 'a list when 'a : equality
```

The function has many forms, one for each instantiation of `'a`:

Instantiating `'a` with `int`:

```
remove 2 [1; 2; 0; 2; 7];;
val it : int list = [1; 0; 7]
```

A polymorphic list function (II)

- A type containing type variables is called a polymorphic type
- The remove function is called a polymorphic function.

```
remove : 'a -> 'a list -> 'a list when 'a : equality
```

The function has many forms, one for each instantiation of `'a`:

Instantiating `'a` with `int`:

```
remove 2 [1; 2; 0; 2; 7];;
val it : int list = [1; 0; 7]
```

Instantiating `'a` with `int list`:

```
remove [2] [[2;1]; [2]; [0;1]; [2]; [5;6;7]];;
val it : int list list = [[2; 1]; [0; 1]; [5; 6; 7]]
```

A polymorphic list function (II)

- A type containing type variables is called a polymorphic type
- The remove function is called a polymorphic function.

```
remove : 'a -> 'a list -> 'a list when 'a : equality
```

The function has many forms, one for each instantiation of 'a:

Instantiating 'a with int:

```
remove 2 [1; 2; 0; 2; 7];;
val it : int list = [1; 0; 7]
```

Instantiating 'a with int list:

```
remove [2] [[2;1]; [2]; [0;1]; [2]; [5;6;7]];;
val it : int list list = [[2; 1]; [0; 1]; [5; 6; 7]]
```

Notice that -> associates to the right:

```
'a -> 'a list -> 'a list  means 'a -> ('a list -> 'a list)
```

Exploiting structured patterns: the isPrefix function

The function `isPrefix` *xs ys* tests whether the list *xs* is a prefix of the list *ys*, for example:

$$\texttt{isPrefix}\ [1;2;3]\ [1;2;3;8;9] \quad = \quad \text{true}$$
$$\texttt{isPrefix}\ [1;2;3]\ [1;2;8;3;9] \quad = \quad \text{false}$$

Exploiting structured patterns: the isPrefix function

The function `isPrefix` *xs ys* tests whether the list *xs* is a prefix of the list *ys*, for example:

$$\texttt{isPrefix}\ [1;2;3]\ [1;2;3;8;9] \quad = \quad \text{true}$$
$$\texttt{isPrefix}\ [1;2;3]\ [1;2;8;3;9] \quad = \quad \text{false}$$

The function is declared as follows:

```
let rec isPrefix xs ys =
   match (xs,ys) with
   | ([],_)               -> true
   | (_,[])               -> false
   | (x::xtail,y::ytail) -> x=y && isPrefix xtail ytail;;

isPrefix [1;2;3] [1;2];;
val it : bool = false
```

Exploiting structured patterns: the isPrefix function

The function `isPrefix` *xs ys* tests whether the list *xs* is a prefix of the list *ys*, for example:

$$\texttt{isPrefix}\ [1;2;3]\ [1;2;3;8;9] \quad = \quad \text{true}$$
$$\texttt{isPrefix}\ [1;2;3]\ [1;2;8;3;9] \quad = \quad \text{false}$$

The function is declared as follows:

```
let rec isPrefix xs ys =
   match (xs,ys) with
   | ([],_)              -> true
   | (_,[])              -> false
   | (x::xtail,y::ytail) -> x=y && isPrefix xtail ytail;;

isPrefix [1;2;3] [1;2];;
val it : bool = false
```

A each clause expresses succinctly a natural property:

Exploiting structured patterns: the isPrefix function

The function `isPrefix` *xs ys* tests whether the list *xs* is a prefix of the list *ys*, for example:

$$\texttt{isPrefix}\ [1;2;3]\ [1;2;3;8;9] \quad = \quad \text{true}$$
$$\texttt{isPrefix}\ [1;2;3]\ [1;2;8;3;9] \quad = \quad \text{false}$$

The function is declared as follows:

```
let rec isPrefix xs ys =
   match (xs,ys) with
   | ([],_)              -> true
   | (_,[])              -> false
   | (x::xtail,y::ytail) -> x=y && isPrefix xtail ytail;;

isPrefix [1;2;3] [1;2];;
val it : bool = false
```

A each clause expresses succinctly a natural property:

- The empty list is a prefix of any list

Exploiting structured patterns: the isPrefix function

The function `isPrefix` *xs ys* tests whether the list *xs* is a prefix of the list *ys*, for example:

$$\texttt{isPrefix}\ [1;2;3]\ [1;2;3;8;9] \quad = \quad \text{true}$$
$$\texttt{isPrefix}\ [1;2;3]\ [1;2;8;3;9] \quad = \quad \text{false}$$

The function is declared as follows:

```
let rec isPrefix xs ys =
   match (xs,ys) with
   | ([],_)              -> true
   | (_,[])              -> false
   | (x::xtail,y::ytail) -> x=y && isPrefix xtail ytail;;

isPrefix [1;2;3] [1;2];;
val it : bool = false
```

A each clause expresses succinctly a natural property:

- The empty list is a prefix of any list
- A non-empty list is not a prefix of the empty list

Exploiting structured patterns: the isPrefix function

The function `isPrefix` *xs ys* tests whether the list *xs* is a prefix of the list *ys*, for example:

$$\texttt{isPrefix}\ [1;2;3]\ [1;2;3;8;9] \quad = \quad \text{true}$$
$$\texttt{isPrefix}\ [1;2;3]\ [1;2;8;3;9] \quad = \quad \text{false}$$

The function is declared as follows:

```
let rec isPrefix xs ys =
   match (xs,ys) with
   | ([],_)              -> true
   | (_,[])              -> false
   | (x::xtail,y::ytail) -> x=y && isPrefix xtail ytail;;

isPrefix [1;2;3] [1;2];;
val it : bool = false
```

A each clause expresses succinctly a natural property:

- The empty list is a prefix of any list
- A non-empty list is not a prefix of the empty list
- A non-empty list (...) is a prefix of another non-empty list (...) if ...

Summary

- Lists

Summary

- Lists
- Polymorphism

Summary

- Lists
- Polymorphism
- Constructors (:: and [] for lists)

Summary

- Lists
- Polymorphism
- Constructors (:: and [] for lists)
- Patterns

Summary

- Lists
- Polymorphism
- Constructors (:: and [] for lists)
- Patterns
- Recursion on the structure of lists

Summary

- Lists
- Polymorphism
- Constructors (:: and [] for lists)
- Patterns
- Recursion on the structure of lists
- Constructors used in patterns to decompose structured values

Summary

- Lists
- Polymorphism
- Constructors (:: and [] for lists)
- Patterns
- Recursion on the structure of lists
- Constructors used in patterns to decompose structured values
- Constructors used in expressions to compose structured values

Summary

- Lists
- Polymorphism
- Constructors (:: and [] for lists)
- Patterns
- Recursion on the structure of lists
- Constructors used in patterns to decompose structured values
- Constructors used in expressions to compose structured values

Exercises

Installation:

- Try to get a working installation and run the power example from the lecture.

Recursive Functions:

- Hansen and Rischel Book (HR):
  Exercise 1.4: Define a function f with $f(n) = 1 + 2 + \ldots + n$.
  Exercise 1.6: Define a function *sum* such that
  $sum(m, n) = m + (m + 1) + ... + (m + n - 1) + (m + n)$
  Exercise 2.8: Define $\binom{n}{k}$ where $0 \leq k \leq n$, according to the definition:

$$\binom{n}{0} = \binom{n}{n} = 1 \qquad \binom{n}{k} = \binom{n-1}{k} + \binom{n-1}{k-1}$$

Lists:

- `memberOf` *x ys* is true iff *x* occurs in the list *ys*
- `insert`(x, ys) where *ys* is an *ordered list*, insert x into the list such that the resulting list is also ordered.
- `sort`(xs) gives a ordered version of *xs*
  - ★ Implement `sort` using `insert`.
  - ★ Expert question (if you have time): what is the worst-case number of steps of this algorithm? How can this be made more efficient?